Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 8

Definición de la variable objetivo: volatilidad futura

Objetivo de la semana. A partir de la base de variables predictoras construida en la Semana 7, definir formalmente la variable objetivo del problema de predicción. Se calculará la volatilidad futura a horizontes de 5, 10 y 20 días utilizando exclusivamente rendimientos posteriores a cada fecha. Se enfatizará la correcta alineación temporal entre predictores y respuesta, la validación manual de las ventanas futuras y la prevención de fuga de información.

Índice

1. Contexto de la Semana 8 **4**

2. Objetivos de aprendizaje **4**

3. Volatilidad histórica y volatilidad futura **5**

4. Horizonte de predicción **6**

5. Interpretación temporal del problema **6**

6. Una línea temporal sencilla **7**

7. Lectura de la base de Semana 7 **8**

8. Validaciones de entrada **9**

9. Construcción computacional de la volatilidad futura **10**

10. Por qué no debemos usar una ventana retrospectiva como objetivo **11**

11. Verificación manual para $h = 5$ **11**

12. Verificación automática para los tres horizontes **12**

13. Valores faltantes al final de la serie **12**

14. Comprobación de que los `NaN` están sólo al final **13**

15. Inspección de las últimas filas **13**

16. Estadística descriptiva de los objetivos **14**

17. Visualización de la volatilidad futura **15**

18. Comparación entre volatilidad histórica y futura **15**

19. Diagrama de dispersión: pasado frente a futuro **16**

20. Persistencia de la volatilidad **17**

21. Comparación entre horizontes futuros **17**

22. Solapamiento entre ventanas futuras **18**

23. Fuga de información: regla central **18**

24. Auditoría simple de columnas objetivo **19**

25. ¿Debe eliminarse ahora toda fila con valores faltantes? **19**

26. Muestra válida sólo para análisis exploratorio **20**

27. Percentiles de volatilidad futura **20**

28. Validaciones automáticas adicionales **21**

28.1. Objetivos no negativos 21

28.2. Número de filas conservado 21

28.3. Columnas originales conservadas 21

28.4. Sólo se agregaron tres objetivos 21

29. Resumen de las variables objetivo **22**

30. Construcción de la base final de la semana **22**

31. Exportación **22**

32. Verificación del archivo exportado **23**

33. Producto esperado **23**

1. Contexto de la Semana 8

En la Semana 7 se construyó una base de variables predictoras a partir de rendimientos, volatilidades históricas, rezagos, medias móviles, momentum, volumen y rango intradiario. El archivo de entrada para esta semana es

`data/processed/SPY_2015_2025_features.csv`.

La característica fundamental de esa base es que cada variable explicativa asociada a una fecha t utiliza únicamente información disponible en t o en fechas anteriores.

Hasta ahora se ha trabajado con cantidades retrospectivas. En esta semana se construirá por primera vez una variable que depende deliberadamente del futuro, pero se utilizará únicamente como *respuesta* o *variable objetivo*.

La secuencia conceptual será

$$\text{información hasta } t \longrightarrow X_t \longrightarrow \text{modelo predictivo} \longrightarrow Y_t,$$

con

$$Y_t = \text{volatilidad observada después de } t.$$

Delimitación de la semana. En esta etapa se construirá y validará la variable objetivo, pero todavía no se ajustarán modelos predictivos. Los modelos base comenzarán en la Semana 9.

2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. distinguir entre volatilidad histórica y volatilidad futura;
2. definir formalmente una variable objetivo para un problema de predicción temporal;
3. interpretar el horizonte de predicción h;
4. construir volatilidad futura a 5, 10 y 20 días;
5. comprender por qué la variable objetivo depende de observaciones posteriores a t;
6. utilizar correctamente `rolling()` y `shift()` para alinear una ventana futura;
7. verificar manualmente que la ventana asociada a una fecha t contiene exactamente los rendimientos $r_{t+1}, \ldots, r_{t+h}$;

8. identificar los valores faltantes estructurales al final de la serie;
9. comprobar que no aparecen valores faltantes interiores inesperados;
10. diferenciar variables explicativas X_t de variables objetivo Y_t;
11. evitar que las columnas objetivo sean utilizadas accidentalmente como predictores;
12. comparar de forma exploratoria la volatilidad reciente con la volatilidad futura;
13. exportar una base con variables predictoras y objetivos correctamente alineados.

3. Volatilidad histórica y volatilidad futura

En la Semana 6 se construyó la volatilidad histórica mediante una ventana móvil. Para una ventana de tamaño w, la volatilidad asociada al tiempo t puede escribirse como

$$\sigma_{t,w} = \sqrt{\frac{1}{w-1} \sum_{i=t-w+1}^{t} (r_i - \bar{r}_{t,w})^2},$$

donde

$$\bar{r}_{t,w} = \frac{1}{w} \sum_{i=t-w+1}^{t} r_i.$$

Por ejemplo,

$$\sigma_{t,5} = \mathrm{sd}\left(r_{t-4}, r_{t-3}, r_{t-2}, r_{t-1}, r_t\right).$$

Esta cantidad utiliza únicamente observaciones disponibles hasta t. Por ello puede formar parte del conjunto de predictores.

Ahora interesa medir la variabilidad que se observará *después* de t. Para un horizonte h, definiremos en este proyecto

$$RV_{t,h} = \sqrt{\frac{1}{h-1} \sum_{i=t+1}^{t+h} \left(r_i - \bar{r}^F_{t,h}\right)^2},$$

donde

$$\bar{r}^F_{t,h} = \frac{1}{h} \sum_{i=t+1}^{t+h} r_i.$$

La letra F recuerda que se trata de una media calculada sobre una ventana *futura*.

Diferencia esencial. La volatilidad histórica $\sigma_{t,w}$ mira hacia atrás y puede utilizarse como predictor. La volatilidad futura $RV_{t,h}$ mira hacia adelante y se utilizará como variable objetivo.

4. Horizonte de predicción

El horizonte h indica cuántas sesiones futuras se utilizarán para medir la volatilidad que queremos predecir.

En esta etapa se considerarán

$$h \in \{5, 10, 20\}.$$

Por tanto, se construirán las columnas

$$\texttt{Target_Vol_5}, \qquad \texttt{Target_Vol_10}, \qquad \texttt{Target_Vol_20}.$$

Sus interpretaciones son

$$\texttt{Target_Vol_5}_t = \mathrm{sd}(r_{t+1}, \ldots, r_{t+5}),$$

$$\texttt{Target_Vol_10}_t = \mathrm{sd}(r_{t+1}, \ldots, r_{t+10}),$$

y

$$\texttt{Target_Vol_20}_t = \mathrm{sd}(r_{t+1}, \ldots, r_{t+20}).$$

Para las primeras comparaciones de modelos se tomará

$$\boxed{\texttt{Target_Vol_5}}$$

como objetivo principal. Los horizontes de 10 y 20 días se conservarán para estudiar posteriormente la sensibilidad de los resultados al horizonte de predicción.

5. Interpretación temporal del problema

Supongamos que nos encontramos al cierre de la sesión t. En ese momento conocemos, entre otras cantidades,

$$r_t, \quad r_{t-1}, \quad r_{t-2}, \ldots,$$

las volatilidades históricas

$$\sigma_{t,5}, \quad \sigma_{t,10}, \quad \sigma_{t,20}, \quad \sigma_{t,30},$$

y los indicadores construidos en la Semana 7.

Toda esa información puede organizarse en un vector de predictores

$$X_t.$$

Sin embargo, todavía no conocemos

$$r_{t+1}, \quad r_{t+2}, \ldots, r_{t+h}.$$

Esos rendimientos sólo se conocerán después. Una vez observados, permiten calcular

$$Y_t = RV_{t,h}.$$

Por tanto, el problema predictivo queda expresado como

$$\boxed{X_t \longrightarrow RV_{t,h}}.$$

Para el horizonte principal,

$$\boxed{X_t \longrightarrow \texttt{Target_Vol_5}_t}.$$

6. Una línea temporal sencilla

Para $h = 5$, la estructura temporal puede representarse como

$$\underbrace{\cdots, r_{t-2}, r_{t-1}, r_t}_{\text{información disponible}} \;\Big|\; \underbrace{r_{t+1}, r_{t+2}, r_{t+3}, r_{t+4}, r_{t+5}}_{\text{ventana objetivo}}.$$

La barra vertical separa el presente del futuro.

La fila correspondiente a la fecha t contendrá simultáneamente:

- predictores construidos con información hasta t;

- una variable objetivo calculada con información posterior a t.

Esto no constituye fuga de información mientras la variable futura se utilice exclusivamente como respuesta y nunca como predictor.

7. Lectura de la base de Semana 7

El notebook de esta semana puede llamarse

`08_variable_objetivo.ipynb`.

Se utilizarán las bibliotecas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

Las rutas principales serán

```
ruta_processed = Path("data/processed")

archivo_features = (
    ruta_processed /
    "SPY_2015_2025_features.csv"
)

archivo_targets = (
    ruta_processed /
    "SPY_2015_2025_targets.csv"
)
```

Antes de leer la base:

```
if not archivo_features.exists():
    raise FileNotFoundError(
        f"No se encontro el archivo: {archivo_features}"
    )
```

Después:

```
datos_features = pd.read_csv(
    archivo_features,
    index_col=0,
    parse_dates=True
)

datos = datos_features.copy(deep=True)
```

Conviene inspeccionar

```
print(datos.shape)
print(datos.index.min())
print(datos.index.max())
print(datos.columns.tolist())

datos.head()
```

8. Validaciones de entrada

La construcción del objetivo depende directamente de la columna `Return`. También deben conservarse algunas variables de semanas anteriores para las comparaciones exploratorias.

```
columnas_necesarias = [
    "Return",
    "Vol_5",
    "Vol_10",
    "Vol_20",
    "Vol_30",
    "Return_Lag_1",
    "Mean_Abs_Return_20",
    "Momentum_20",
    "Range_HL",
    "Ratio_Vol_5_30"
]

faltantes = [
    col for col in columnas_necesarias
    if col not in datos.columns
]

assert len(faltantes) == 0, (
    f"Faltan columnas: {faltantes}"
)
```

También se verifica la estructura temporal:

```
assert datos.index.is_monotonic_increasing
assert datos.index.is_unique
```

Y la integridad de los rendimientos:

```
assert not datos["Return"].isna().any()
assert np.isfinite(datos["Return"]).all()
```

Estas pruebas son importantes porque un valor faltante interior en `Return` afectaría varias ventanas futuras consecutivas.

9. Construcción computacional de la volatilidad futura

Se utilizarán los horizontes

```
horizontes = [5, 10, 20]
```

La construcción puede realizarse mediante

```
for h in horizontes:
    datos[f"Target_Vol_{h}"] = (
        datos["Return"]
        .rolling(window=h)
        .std(ddof=1)
        .shift(-h)
    )
```

La instrucción tiene dos partes que deben comprenderse por separado.

Primero,

```
datos["Return"].rolling(window=h).std(ddof=1)
```

calcula, en una posición j, la desviación estándar de

$$r_{j-h+1}, \ldots, r_j.$$

Después,

```
.shift(-h)
```

desplaza ese valor h posiciones hacia atrás. De esta manera, en la posición t queda almacenada la desviación estándar calculada en $t + h$, cuya ventana es

$$r_{t+1}, \ldots, r_{t+h}.$$

Por tanto,

$$\texttt{Target_Vol_h}_t = \mathrm{sd}(r_{t+1}, \ldots, r_{t+h}).$$

Importante. El desplazamiento negativo está permitido aquí porque estamos construyendo deliberadamente la variable objetivo. No debe utilizarse este procedimiento para crear variables explicativas.

10. Por qué no debemos usar una ventana retrospectiva como objetivo

El siguiente código

```
datos["Objetivo_incorrecto"] = (
    datos["Return"]
    .rolling(window=5)
    .std(ddof=1)
)
```

no construye la volatilidad de los siguientes cinco días. En la fecha t produce

$$\mathrm{sd}\left(r_{t-4}, r_{t-3}, r_{t-2}, r_{t-1}, r_t\right),$$

que es una medida retrospectiva.

Tampoco conviene usar

```
datos["Return"].shift(-1).rolling(5).std()
```

sin analizar cuidadosamente la alineación, porque la combinación de desplazamiento y ventana puede resultar menos transparente y facilitar errores de índice.

En este proyecto se utilizará una construcción que pueda verificarse directamente mediante una ventana manual.

11. Verificación manual para $h = 5$

Una de las pruebas más importantes de esta semana consiste en seleccionar una observación interior y comprobar que el objetivo contiene exactamente los cinco rendimientos posteriores.

```
i = 100
h = 5

ventana_futura = (
    datos["Return"]
    .iloc[i + 1:i + h + 1]
)

objetivo_manual = ventana_futura.std(ddof=1)
objetivo_python = datos["Target_Vol_5"].iloc[i]

print("Fecha t:", datos.index[i])
print("Fechas futuras:")
print(ventana_futura.index)
print("Manual:", objetivo_manual)
print("Python:", objetivo_python)

```

```
assert len(ventana_futura) == h
assert np.isclose(
    objetivo_manual,
    objetivo_python
)
```

La ventana debe comenzar en $i + 1$ y terminar en $i + h$. El rendimiento de la propia fecha i no debe formar parte del objetivo.

12. Verificación automática para los tres horizontes

La misma comprobación puede automatizarse:

```
i = 100

for h in horizontes:
    ventana = (
        datos["Return"]
        .iloc[i + 1:i + h + 1]
    )

    esperado = ventana.std(ddof=1)
    obtenido = datos[f"Target_Vol_{h}"].iloc[i]

    assert len(ventana) == h
    assert np.isclose(esperado, obtenido)
```

Para aumentar la confianza en la alineación, conviene repetir la prueba en varias posiciones alejadas del final de la serie:

```
posiciones = [50, 100, 250, 500]

for i in posiciones:
    for h in horizontes:
        ventana = (
            datos["Return"]
            .iloc[i + 1:i + h + 1]
        )

        esperado = ventana.std(ddof=1)
        obtenido = datos[f"Target_Vol_{h}"].iloc[i]

        assert np.isclose(esperado, obtenido)
```

13. Valores faltantes al final de la serie

La volatilidad futura no puede calcularse para las últimas observaciones porque no existen suficientes datos posteriores dentro de la muestra.

Para $h = 5$, las últimas cinco filas no tienen una ventana futura completa. Para $h = 10$, faltan las últimas diez, y para $h = 20$, las últimas veinte.

Por tanto, si `Return` no contiene valores faltantes, esperamos

$$\begin{aligned} \texttt{Target_Vol_5}: &\quad 5 \text{ valores faltantes,} \\ \texttt{Target_Vol_10}: &\quad 10 \text{ valores faltantes,} \\ \texttt{Target_Vol_20}: &\quad 20 \text{ valores faltantes.} \end{aligned}$$

La comprobación será

```python
for h in horizontes:
    col = f"Target_Vol_{h}"

    n_nan = datos[col].isna().sum()
    print(col, n_nan)

    assert n_nan == h
```

Interpretación. Estos valores faltantes finales no representan un problema de calidad de datos. Son valores faltantes estructurales producidos por la definición de una ventana futura.

14. Comprobación de que los `NaN` están sólo al final

No basta con contar los valores faltantes. También debemos verificar su posición.

```python
n = len(datos)

for h in horizontes:
    col = f"Target_Vol_{h}"

    parte_valida = datos[col].iloc[:n-h]
    cola = datos[col].iloc[n-h:]

    assert not parte_valida.isna().any()
    assert cola.isna().all()
```

Con esta prueba se detectaría, por ejemplo, un valor faltante inesperado en la parte central de la serie.

15. Inspección de las últimas filas

Conviene observar directamente el final de la base:

```
datos[
    [
        "Return",
        "Target_Vol_5",
        "Target_Vol_10",
        "Target_Vol_20"
    ]
].tail(25)
```

La estructura esperada es escalonada:

- `Target_Vol_20` comienza a ser indefinida veinte observaciones antes del final;
- `Target_Vol_10` comienza diez observaciones antes;
- `Target_Vol_5` comienza cinco observaciones antes.

16. Estadística descriptiva de los objetivos

Una primera descripción puede obtenerse mediante

```
columnas_target = [
    "Target_Vol_5",
    "Target_Vol_10",
    "Target_Vol_20"
]

resumen_targets = (
    datos[columnas_target]
    .describe()
    .T
)

resumen_targets
```

Interesa comparar, entre otras cantidades:

- media;
- desviación estándar;
- mediana;
- percentiles 25 y 75;
- máximos observados.

Los tres objetivos representan el mismo fenómeno en escalas temporales diferentes. Por ello sus distribuciones no tienen por qué ser idénticas.

17. Visualización de la volatilidad futura

Podemos representar los tres horizontes simultáneamente:

```
fig, ax = plt.subplots(figsize=(12, 5))

for h in horizontes:
    ax.plot(
        datos.index,
        datos[f"Target_Vol_{h}"],
        label=f"Volatilidad futura {h} dias",
        linewidth=1
    )

ax.set_title(
    "Volatilidad futura para distintos horizontes"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Volatilidad")
ax.legend()

plt.show()
```

Las ventanas más cortas suelen reaccionar con mayor rapidez a cambios bruscos, mientras que las ventanas más largas producen medidas más suavizadas.

18. Comparación entre volatilidad histórica y futura

Una comparación natural es

$$\sigma_{t,5} \qquad \text{frente a} \qquad RV_{t,5}.$$

La primera resume los cinco rendimientos más recientes hasta t; la segunda resume los cinco rendimientos inmediatamente posteriores.

```
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    datos.index,
    datos["Vol_5"],
    label="Volatilidad historica 5 dias",
    linewidth=1
)

ax.plot(
    datos.index,
    datos["Target_Vol_5"],
    label="Volatilidad futura 5 dias",
    linewidth=1
```

```
)

ax.set_title(
    "Volatilidad historica y volatilidad futura"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Volatilidad")
ax.legend()

plt.show()
```

No se espera que ambas curvas coincidan. Precisamente el objetivo de las siguientes semanas será estudiar hasta qué punto la información disponible en t permite anticipar la volatilidad posterior.

19. Diagrama de dispersión: pasado frente a futuro

Podemos estudiar la asociación entre

$$\texttt{Vol_5}_t \quad \text{y} \quad \texttt{Target_Vol_5}_t.$$

```
comparacion = (
    datos[
        ["Vol_5", "Target_Vol_5"]
    ]
    .dropna()
)

fig, ax = plt.subplots(figsize=(6, 6))

ax.scatter(
    comparacion["Vol_5"],
    comparacion["Target_Vol_5"],
    alpha=0.4
)

ax.set_title(
    "Volatilidad reciente frente a volatilidad futura"
)
ax.set_xlabel("Vol_5")
ax.set_ylabel("Target_Vol_5")

plt.show()
```

La correlación lineal correspondiente puede calcularse mediante

```
correlacion_5 = (
    comparacion["Vol_5"]
    .corr(comparacion["Target_Vol_5"])
)
```

```
5
6 print("Correlacion:", correlacion_5)
```

Esta correlación es únicamente un análisis exploratorio. No constituye todavía una evaluación predictiva fuera de muestra.

20. Persistencia de la volatilidad

Una característica común de muchas series financieras es que periodos de movimientos grandes tienden a agruparse temporalmente. Este fenómeno suele denominarse *clustering* o agrupamiento de volatilidad.

La comparación entre `Vol_5` y `Target_Vol_5` permite observar de manera preliminar si episodios de elevada volatilidad reciente tienden a estar acompañados por elevada volatilidad en los días posteriores.

Sin embargo, una asociación visual o una correlación positiva no garantiza capacidad predictiva. Para evaluar predicción será necesario respetar la estructura temporal de entrenamiento y prueba, tarea que se desarrollará en semanas posteriores.

21. Comparación entre horizontes futuros

También podemos estudiar la relación entre los tres objetivos:

```
correlaciones_target = (
    datos[columnas_target]
    .corr()
)

correlaciones_target
```

Y representar, por ejemplo,

```
fig, ax = plt.subplots(figsize=(6, 6))

muestra = datos[
    ["Target_Vol_5", "Target_Vol_20"]
].dropna()

ax.scatter(
    muestra["Target_Vol_20"],
    muestra["Target_Vol_5"],
    alpha=0.4
)

ax.set_title(
    "Volatilidad futura: horizonte 5 vs 20 dias"
)
```

```
ax.set_xlabel("Target_Vol_20")
ax.set_ylabel("Target_Vol_5")

plt.show()
```

Los horizontes comparten observaciones futuras y, por ello, pueden presentar correlaciones elevadas. No deben interpretarse como tres variables independientes.

22. Solapamiento entre ventanas futuras

Existe una característica temporal importante que debe comprenderse antes de modelar.

Para $h = 5$, el objetivo en t utiliza

$$r_{t+1}, r_{t+2}, r_{t+3}, r_{t+4}, r_{t+5},$$

mientras que el objetivo en $t + 1$ utiliza

$$r_{t+2}, r_{t+3}, r_{t+4}, r_{t+5}, r_{t+6}.$$

Ambas ventanas comparten cuatro observaciones. Por tanto, las variables objetivo consecutivas están construidas a partir de ventanas fuertemente solapadas.

Consecuencia metodológica. El solapamiento no es un error y se conservará en este proyecto. Sin embargo, explica parte de la dependencia temporal entre observaciones consecutivas y refuerza la necesidad de no utilizar particiones aleatorias de entrenamiento y prueba.

23. Fuga de información: regla central

Después de agregar las columnas objetivo, cada fila contendrá tanto variables conocidas en t como resultados futuros. Esto exige una separación conceptual estricta.

Definiremos

$$\mathcal{X} = \{\text{columnas utilizables como predictores}\}$$

y

$$\mathcal{Y} = \{\texttt{Target_Vol_5}, \texttt{Target_Vol_10}, \texttt{Target_Vol_20}\}.$$

Las columnas de $\mathcal{Y}$ nunca deben aparecer dentro de la matriz de predictores.

Por ejemplo, más adelante será incorrecto hacer simplemente

```
X = datos.select_dtypes(include="number")
```

porque esta instrucción podría incluir accidentalmente las columnas objetivo.

Una práctica más segura será construir explícitamente la lista de predictores o eliminar de manera explícita todas las columnas que no puedan utilizarse como entradas.

24. Auditoría simple de columnas objetivo

Podemos definir desde ahora

```
columnas_objetivo = [
    "Target_Vol_5",
    "Target_Vol_10",
    "Target_Vol_20"
]
```

Y verificar su presencia:

```
for col in columnas_objetivo:
    assert col in datos.columns
```

Cuando se construya la matriz X en semanas posteriores, se deberá comprobar

```
for col in columnas_objetivo:
    assert col not in X.columns
```

Esta prueba sencilla evita una de las formas más graves de fuga de información.

25. ¿Debe eliminarse ahora toda fila con valores faltantes?

Todavía no es necesario eliminar todas las filas incompletas de la base maestra.

Las variables predictoras contienen valores faltantes estructurales al inicio por efecto de rezagos y ventanas móviles. Las variables objetivo contienen valores faltantes estructurales al final por efecto del horizonte futuro.

Para un objetivo específico, por ejemplo `Target_Vol_5`, la muestra utilizable deberá satisfacer simultáneamente:

- predictores requeridos disponibles;
- objetivo disponible;
- valores finitos en todas las variables del modelo.

La muestra exacta dependerá de qué predictores utilice cada modelo. Por ello se conservará por ahora la base completa.

> **Decisión del proyecto.** La Semana 8 exportará la base completa con los valores faltantes estructurales. La selección de la muestra de modelado se realizará cuando se construyan formalmente X e y.

26. Muestra válida sólo para análisis exploratorio

Cuando sea necesario calcular estadísticas o gráficas específicas del objetivo principal, puede construirse temporalmente una muestra válida:

```
exploracion_5 = datos[
    [
        "Vol_5",
        "Vol_20",
        "Mean_Abs_Return_20",
        "Range_HL",
        "Target_Vol_5"
    ]
].dropna()
```

Esta base se utiliza únicamente para análisis exploratorio. No representa todavía la partición definitiva de entrenamiento, validación y prueba.

27. Percentiles de volatilidad futura

Como análisis descriptivo puede calcularse el percentil 75 de la volatilidad futura de cinco días:

```
umbral_75 = (
    datos["Target_Vol_5"]
    .dropna()
    .quantile(0.75)
)

print("Percentil 75:", umbral_75)
```

También podemos identificar, de manera exclusivamente descriptiva, qué fechas presentan una volatilidad futura particularmente alta:

```
episodios_altos = datos.loc[
    datos["Target_Vol_5"] > umbral_75,
    ["Target_Vol_5", "Vol_5", "Vol_20"]
]

episodios_altos.head()
```

No se construirá todavía una variable de clasificación. Esta posibilidad se retomará más adelante si se desea estudiar eventos de alta volatilidad como problema complementario.

28. Validaciones automáticas adicionales

Conviene convertir las propiedades esenciales de la variable objetivo en pruebas automáticas.

28.1. Objetivos no negativos

La desviación estándar no puede ser negativa:

```
for col in columnas_objetivo:
    valores = datos[col].dropna()

    assert (valores >= 0).all()
    assert np.isfinite(valores).all()
```

28.2. Número de filas conservado

La construcción del objetivo no debe modificar el calendario:

```
assert (
    datos.shape[0]
    == datos_features.shape[0]
)

assert (
    datos.index
    .equals(datos_features.index)
)
```

28.3. Columnas originales conservadas

```
for col in datos_features.columns:
    assert col in datos.columns
```

28.4. Sólo se agregaron tres objetivos

```
columnas_nuevas = [
    col for col in datos.columns
    if col not in datos_features.columns
]

assert columnas_nuevas == columnas_objetivo
```

29. Resumen de las variables objetivo

Las variables construidas en esta semana pueden resumirse como sigue:

Variable	Horizonte	Información utilizada
`Target_Vol_5`	5 sesiones	$r_{t+1}, \ldots, r_{t+5}$
`Target_Vol_10`	10 sesiones	$r_{t+1}, \ldots, r_{t+10}$
`Target_Vol_20`	20 sesiones	$r_{t+1}, \ldots, r_{t+20}$

La variable principal para iniciar la comparación de modelos será `Target_Vol_5`.

30. Construcción de la base final de la semana

La base final conservará todas las variables de la Semana 7 y añadirá los tres objetivos:

```
datos_targets = datos.copy()
```

Antes de exportar:

```
assert (
    datos_targets.shape[0]
    == datos_features.shape[0]
)

assert (
    datos_targets.shape[1]
    == datos_features.shape[1] + 3
)

assert (
    datos_targets.index
    .equals(datos_features.index)
)
```

El pipeline queda ahora

$$\texttt{raw} \longrightarrow \texttt{clean} \longrightarrow \texttt{returns} \longrightarrow \texttt{volatility} \longrightarrow \texttt{features} \longrightarrow \texttt{targets}.$$

31. Exportación

La nueva base se almacenará mediante

```
datos_targets.to_csv(
    archivo_targets,
```

```
    index=True
)
```

El archivo resultante será

`data/processed/SPY_2015_2025_targets.csv`.

32. Verificación del archivo exportado

Después de guardar, se leerá nuevamente:

```
verificacion = pd.read_csv(
    archivo_targets,
    index_col=0,
    parse_dates=True
)
```

Se verifican dimensiones, índice y columnas objetivo:

```
assert (
    verificacion.shape
    == datos_targets.shape
)

assert (
    verificacion.index
    .equals(datos_targets.index)
)

assert verificacion.index.is_monotonic_increasing
assert verificacion.index.is_unique

for col in columnas_objetivo:
    assert col in verificacion.columns
```

También conviene repetir la comprobación del número de valores faltantes:

```
for h in horizontes:
    col = f"Target_Vol_{h}"
    assert verificacion[col].isna().sum() == h
```

33. Producto esperado

Al finalizar la semana se deberán tener los siguientes productos:

1. notebook `08_variable_objetivo.ipynb`;

2. base `data/processed/SPY_2015_2025_targets.csv`;
3. definición matemática de volatilidad futura para $h = 5, 10, 20$;
4. tres columnas objetivo correctamente alineadas en el tiempo;
5. verificaciones manuales y automáticas de las ventanas futuras;
6. análisis del patrón de valores faltantes finales;
7. estadísticas descriptivas de los tres objetivos;
8. gráficas de volatilidad futura para diferentes horizontes;
9. comparación exploratoria entre volatilidad histórica y futura;
10. documentación explícita de la separación entre predictores X_t y respuesta Y_t.

Conexión con la Semana 9. La base construida en esta semana contiene tanto las variables explicativas disponibles hasta cada fecha como las medidas de volatilidad futura que se desean predecir. En la Semana 9 se utilizará `Target_Vol_5` como objetivo principal para implementar modelos base sencillos y establecer una referencia de desempeño antes de introducir EWMA, GARCH y modelos de aprendizaje automático.